Lesson plan · for the teacher

Teaching debugging

Programming · Level 3 · Lesson 3

Time
40 min

Materials
Three buggy programs on paper; Poster paper and markers; One phone to run the programs

Learning goals

- Teach a debugging routine.
- Read Python's error messages with a class.
- Use printing values and explaining aloud as tools.

Activity: Bug detectives

1. Warm-up (5 min): Show an error message on the board. Read it aloud together: the kind, the line, the words. Ask what the pupil should look at first.
2. Teach (10 min): Write the routine: read, find the line (and the one before), guess, check, change one thing, test. Explain the four common errors with an example each.
3. Hunt (15 min): Pairs get three buggy programs on paper, one with no error message. They follow the routine and trace values in a table for the silent bug.
4. Share (5 min): Pairs say which step found each bug. Praise the routine, not the speed.
5. Wrap-up (5 min): Turn the routine into a wall poster together. Then give the exit ticket.

Hunting bugs on paper works with no devices; run the fixed programs on one phone at the end.

Discussion

- Why is fixing a pupil's bug for them a missed chance?
- How would you teach a bug that gives no error?
- How can we make bugs feel normal in our class?

Exit ticket

- Write the question you will ask the next pupil who says "it doesn't work".

Try it at school tomorrow
Next time a pupil says "it doesn't work", don't touch their code. Ask: "What did you expect? What happened? Read me the error."

Practice questions · for you

Teaching debugging

Programming · Level 3 · Lesson 3

1 **A pupil's loop prints the wrong total, with no error. What is the best thing to say first?**

- ◯ "Line 3 is wrong; write it like this."
- ◯ "What should total be after the first time round?"
- ◯ "Delete it and start again."

2 **Match each error to what it usually means.**

SyntaxError	• • • •	Mixing text and numbers
NameError	• • • •	Python can't read the line: a missing colon, bracket or quote
TypeError	• • • •	Lines not lined up under if, for or def
IndentationError	• • • •	A name that was never made, often a typo

3 **This program should print the total of the marks, 36, but prints 9. There is no error message. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  marks = [12, 15, 9]
2  total = 0
3  for m in marks:
4      total = m
5  print(total)
```

4 **Fix this program: it should print how many pupils are absent, like Absent: 3, but it stops with a TypeError.**

```
pupils = int(input("Pupils? "))
present = input("Present? ")
absent = pupils - present
print("Absent:", absent)
```

Write your program:

5 **This program should print the total once, Total: 18, but it prints three lines. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  total = 0
2  for m in [4, 6, 8]:
3      total = total + m
4      print("Total:", total)
```

✎ **Think of the last time a pupil said "It doesn't work!". What did you do? Write the question you would ask now instead.**

Check your answers

Teaching debugging

Programming · Level 3 · Lesson 3

1 **"What should total be after the first time round?"**

A question sends the pupil back to trace their own program, and teaches the move they need next time.

2 **SyntaxError — Python can't read the line: a missing colon, bracket or quote**
NameError — A name that was never made, often a typo
TypeError — Mixing text and numbers
IndentationError — Lines not lined up under if, for or def

Knowing what each kind of error usually means turns a scary message into a clue.

3 **Line 4:** `total = total + m`

total = m replaces the total with each mark, so only the last one, 9, is left.

4

```
pupils = int(input("Pupils? "))
present = int(input("Present? "))
absent = pupils - present
print("Absent:", absent)
```

The TypeError names line 3, where text is subtracted from a number, but the cause is line 2: input() gives text.

5 **Line 4:** `print("Total:", total)`

The print is indented, so it belongs to the loop and runs every turn.